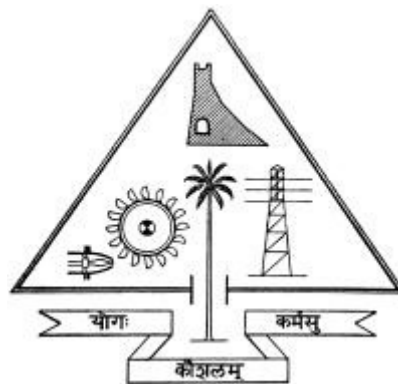


C2SAFERUST-LLM: INTERPROCEDURAL OWNERSHIP ANALYSIS FOR SAFE C-TO-RUST TRANSPILATION

submitted by

**GOKULKRISHNA A K
(TCR25MCA-2028)**



**DEPARTMENT OF COMPUTER APPLICATIONS
GOVERNMENT ENGINEERING COLLEGE
THRISSUR - 680009**

OCTOBER 2026

CONTENTS

List of Figures	iii
List of Tables	iv
List of Abbreviations	v
1 INTRODUCTION	1
2 LITERATURE REVIEW	5
3 ENVIRONMENTAL STUDY	9
3.1 System Configuration	9
3.1.1 Hardware Requirements	10
3.1.2 Software Requirements	10
3.1.3 Software Requirements for Development	11
3.2 Software Specification	12
3.2.1 Rust and Cargo	12
3.2.2 rustc and Structured Diagnostics	12
3.2.3 petgraph	12
3.2.4 serde	13
3.2.5 request	13
3.2.6 egui	13
4 SYSTEM ANALYSIS	15
4.1 Requirements Analysis	15
4.2 Existing System	16
4.3 Limitations of Existing System	16
4.4 Proposed System	18
4.5 Advantages of Proposed System	20
4.6 Functional Requirements	20
4.6.1 Input File Acceptance and Validation	21
4.6.2 Preprocessor Expansion	21
4.6.3 AST Construction	21
4.6.4 Multi-Level Pointer Chain Analysis	21
4.6.5 Union-to-Enum Transformation	22
4.6.6 Rust Code Generation	22
4.6.7 Call Graph Construction	22
4.6.8 Interprocedural Alias and Points-to Analysis	22
4.6.9 Ownership-Transfer Summaries for Multi-Level Pointers	23
4.6.10 Cross-Function Union Variant Tracking	23
4.6.11 Fallback for Data-Dependent Ownership	23
4.6.12 LLM-Guided Repair Agent	23
4.7 Non-Functional Requirements	24
4.7.1 Scalability	24

4.7.2	Backward Compatibility	24
4.7.3	Transparency	24
4.7.4	Maintainability	25
4.8	Feasibility Study	25
4.8.1	Technical Feasibility	25
4.8.2	Operational Feasibility	25
4.8.3	Economical Feasibility	26
5	SYSTEM DESIGN	27
5.1	Application Architecture	27
5.2	Data Flow Diagram	30
5.2.1	DFD Level 0	30
5.2.2	DFD Level 1	30
5.2.3	DFD Level 2	32
5.2.3.1	Expansion of Process 0.3: Per-Function Semantic Analysis	32
5.2.3.2	Expansion of Process 0.4: Interprocedural Analysis	34
5.2.3.3	Expansion of Process 0.6: Compiler Validation and LLM-Guided Repair	36
5.3	List of Modules	37
5.3.1	Input Acceptance and Validation Module	37
5.3.2	Preprocessing and AST Construction Module	37
5.3.3	Per-Function Semantic Analysis Module	37
5.3.4	Interprocedural Analysis Module	38
5.3.5	Ownership Classification and Code Generation Module	38
5.3.6	Compiler Validation and LLM-Guided Repair Module	38
5.3.7	Report Assembly and Output Display Module	38
	Bibliography	40

LIST OF FIGURES

5.1	C2SafeRust-LLM — Block Architecture	29
5.2	Data Flow Diagram Level 0 (Context Diagram)	30
5.3	Data Flow Diagram Level 1	31
5.4	Data Flow Diagram Level 2 — Expansion of Process 0.3	33
5.5	Data Flow Diagram Level 2 — Expansion of Process 0.4	35
5.6	Data Flow Diagram Level 2 — Expansion of Process 0.6	36

LIST OF TABLES

4.1	Traceability from idiom to the new component that is insufficient if scoped per-function, and the interprocedural mechanism proposed to resolve it	19
-----	--	----

LIST OF ABBREVIATIONS

API	– Application Programming Interface
AST	– Abstract Syntax Tree
CI/CD	– Continuous Integration / Continuous Deployment
IDE	– Integrated Development Environment
JSON	– JavaScript Object Notation
LLM	– Large Language Model
MCA	– Master of Computer Applications
POSIX	– Portable Operating System Interface
RAM	– Random Access Memory
SRS	– Software Requirements Specification
URL	– Uniform Resource Locator
VS Code	– Visual Studio Code

CHAPTER 1

INTRODUCTION

The majority of operating system kernels, embedded firmware, and network-facing daemons in active use today are still written in C. The language survives because it gives programmers direct, predictable control over memory and hardware, but that same permissiveness is the root cause of a large share of reported security vulnerabilities: buffer overflows, use-after-free defects, dangling-pointer dereferences, and data races all trace back to the absence of any compiler-enforced discipline around memory ownership. Rust addresses this gap directly. Matsakis and Klock [3] and Klabnik and Nichols [1] establish that Rust’s type system encodes ownership and borrowing rules that are checked exhaustively at compile time, so an entire category of memory-safety and data-race bugs is rejected before the program ever runs. This guarantee, however, is only as strong as the code that is written to satisfy it; Qin et al. [2] show, through empirical studies of real-world Rust codebases, that safety issues still concentrate around unsafe blocks introduced at API and foreign-function boundaries, which is precisely the kind of code that a naive C-to-Rust translator tends to produce in bulk. This has motivated a growing body of work on automated C-to-Rust translation that goes beyond mechanical, unsafe-block-laden mappings and instead infers genuine Rust ownership from C source.

The predecessor system to this project, C2SafeRust, took a compiler-in-the-loop approach to this problem. It parses a C source file into an internal abstract syntax tree, builds a semantic model of def-use chains, pointer aliasing, and memory-allocation patterns, classifies every variable into one of eight ownership categories, generates a candidate Rust translation, and

then treats the structured JSON diagnostics emitted by `rustc` as a machine-readable repair specification rather than a terminal error report. Evaluated on a structured ten-program benchmark suite exercising scalar expressions, one-dimensional arrays, struct manipulation, and recursion, the intra-procedural system achieved an 80% compilation success rate and produced zero unsafe blocks in 75% of the successfully translated programs, converging in an average of 2.3 repair iterations. These figures describe the existing intra-procedural baseline only; they are reported here to establish the starting point for the present work and are not claims made about the interprocedural extension described in this report, which has not yet been implemented or measured.

This project proposes C2SafeRust-LLM, a single extension to the base C2SafeRust pipeline that simultaneously (i) widens the accepted C subset by adding a preprocessor expansion stage, a multi-level pointer chain analyser, and a union-to-enum transformer, (ii) adds a new interprocedural analysis pass, and (iii) replaces the fixed rule-based repair engine with an LLM-guided repair agent capable of generalising across compiler error patterns a rule-based engine can never fully enumerate. These five additions are co-designed rather than staged: the interprocedural pass is included from the outset because a per-function analysis of the newly introduced multi-level-pointer and union constructs cannot, even in principle, resolve six identified idioms detailed in Chapter 4. All five components trace back to the same structural limitation in C2SafeRust: every ownership decision – whether a pointer is classified as a borrow, a unique heap owner, or a shared reference – is made by analysing one function body in isolation. Its semantic model is built per function, and no information about how a variable is used at its call sites, or how it flows through the bodies of the functions it is passed to, is available when the ownership classifier runs.

This report presents the design of an interprocedural extension to this pipeline. The extension is motivated by six concrete C idioms, all of them common in real systems code, whose correct translation is provably impossible

from within a single function body: aliased mutable parameters that only alias at a particular call site; ownership transferred out of the caller through a `T**` argument; ownership transferred into the caller through a `T**` out-parameter; overlapping pointer-arithmetic slices that are only overlapping for specific call-site arguments; a union's active variant, which is set in one function and read in another; and, in the hardest case, ownership that depends on a value known only at run time. For each of the first five idioms, resolving the ambiguity requires information that exists only at the call site or inside the body of a different function than the one being translated – that is, it requires an interprocedural analysis. The sixth idiom is included deliberately because it cannot be fully resolved even by an interprocedural pass; it establishes the boundary of what static analysis can achieve here and the point at which the LLM-guided repair agent introduced by this project remains the appropriate fallback rather than being made redundant by this extension.

The objective of this project is therefore to extend the base C2SafeRust pipeline with all five bundled components at once, chief among them a new interprocedural analysis pass positioned after per-function semantic analysis and before ownership classification, that constructs a whole-program call graph over direct calls and propagates alias and ownership information across call boundaries as function summaries before the code generator commits to a Rust type for any pointer parameter. Concretely, the revised pipeline proceeds through six stages in sequence: input validation; AST processing, now preceded by a preprocessor expansion step that resolves macros and conditional-compilation directives so a wider C subset can be parsed at all; per-function semantic analysis, extended with the new multi-level pointer chain analyser and union-to-enum transformer; the new interprocedural analysis stage – where the call graph and the alias and ownership summaries are built; Rust code generation; and finally the compiler loop and LLM-guided repair stage that produces the final Rust code and report. The `rustc` JSON diagnostic feedback loop from the compiler loop back to the interprocedural analysis stage reuses the same compiler-invocation mechanism as the base

system, but the repair engine consuming those diagnostics is replaced by the LLM-guided repair agent described above, remaining available as a fallback for cases the static passes cannot resolve; the new interprocedural pass sits upstream of code generation so that call-site-dependent ownership decisions are resolved before a repair iteration is ever needed, rather than being discovered only after `rustc` rejects the generated code.

In scope, this means adding a call-graph construction step, an interprocedural points-to and alias analysis over call-site arguments, and ownership-transfer summaries that link a callee’s write-through pointer parameters back to the caller’s local variables. Out of scope, deliberately, is any attempt to make the sixth idiom fully static; the design instead routes that case to a conservative `Option<Box<T>>` fallback or to the LLM-guided repair agent introduced by this project, consistent with the position that interprocedural analysis narrows the class of problems the repair agent must handle rather than eliminating the need for it.

The remainder of this report is organised as follows. Chapter 2 reviews the literature on ownership-guided and compiler-feedback-driven C-to-Rust translation and situates the interprocedural ownership-transfer gap within it. Chapter 3 describes the environment – hardware, software, and Rust ecosystem components – required to build and evaluate the extension. Chapter 4 analyses the requirements of the proposed system as a whole in detail, including why C2SafeRust’s strictly intra-procedural design cannot be extended to support the six target idioms without an interprocedural analysis pass, and the functional and non-functional requirements that follow for all five bundled components.

CHAPTER 2

LITERATURE REVIEW

Automated migration of legacy C codebases to memory-safe Rust has become an active research area since Matsakis and Klock [3] first formalised Rust’s ownership and borrowing model as a static, compile-time alternative to garbage collection and manual memory management. Klabnik and Nichols [1] document, in the standard reference on Rust’s design, how the borrow checker eliminates data races and use-after-free errors without any runtime cost, establishing the appeal of the language for systems software. What is less well established, and what this chapter argues is the central open problem for automated translation tools specifically, is *where* the information needed to make a correct ownership decision actually resides. This chapter reviews the literature on ownership-guided C-to-Rust translation, empirical studies of translation limits, targeted structural refactoring, and post-hoc repair, and organises the discussion around a single unifying thread: an interprocedural ownership-transfer gap that recurs across every category of prior work, in one form or another.

Empirical evidence for why this gap matters even after translation succeeds comes from studies of real-world Rust code rather than translated code. Qin et al. [2] analyse memory- and thread-safety practices across a large corpus of published Rust crates and find that safety violations do not disappear once code is written in Rust; they concentrate at unsafe boundaries, in foreign-function interfaces, and in places where the type system’s guarantees are deliberately bypassed to express a pattern the checker cannot otherwise accept. This is directly relevant to translation tools: every C pointer pattern that a translator cannot classify with confidence

becomes exactly this kind of bypass, either an unsafe block or a translation failure. Minimising these bypasses is therefore not a cosmetic goal but a safety-relevant one, and it depends on how much of the true ownership behaviour of the source program the translator can actually observe.

The first generation of ownership-guided translators addressed this by strengthening intra-procedural analysis. Emre et al. [4] introduce Laertes, which treats rustc’s own diagnostic output as a specification of what the borrow checker requires and iteratively rewrites raw-pointer types in response, rather than relying on a separate static ownership model. This compiler-feedback strategy is the same one the base C2SafeRust pipeline builds on, and it is effective for the common case where a pointer’s correct type can be determined from how it is used within the function that declares it. Its limits, however, are exactly the limits of a per-function view: Emre et al. [5] measure, in a later and considerably larger empirical study by an overlapping set of authors, how far aliasing analysis alone can go in resolving raw pointers to safe references, tracing most of the residual imprecision to the use of equality-based alias equivalence classes rather than finer, call-aware reasoning about how arguments alias across a call boundary. This is, in effect, an empirical demonstration of the interprocedural ownership-transfer gap from the inside of a working system: the aliasing questions that remain unresolved are disproportionately the ones that only become decidable once the analysis is allowed to look at the caller and the callee together, which is exactly the class of case illustrated by aliased mutable parameters and pointer-arithmetic slices that overlap only at specific call sites.

A second line of work accepts that some ownership patterns cannot be inferred generically and instead targets specific, well-characterised idioms with dedicated structural transformations. Hong and Ryu [6] replace C’s output-parameter idiom – a function that writes its result through a pointer argument rather than returning it – with an algebraic data type such as `Option` or `Result` in the generated Rust, eliminating an entire class of write-through-pointer translations by recognising the idiom syntactically. This is

close in spirit to the extension proposed in this report, in that it is explicitly interprocedural: the transformation only makes sense once the analysis understands what the function's callers do with the value written through the pointer. Its scope, however, is narrower than the general problem. It resolves one of the six target idioms considered here – ownership created through a `T**` out-parameter – but does not generalise to the mirror-image case where ownership is instead consumed through a `T**` argument, nor to aliased mutable parameters or overlapping slices. Recovery of production-realistic C constructs more broadly has also been addressed at the preprocessor level; Peng et al. [7] show, with a modular wrapper for translating C macros and conditional compilation directives to native Rust attributes, that macro-heavy headers can be recovered without abandoning the underlying translation pipeline. This work is complementary rather than competing: preprocessor recovery widens the C subset a translator can parse at all, but it operates before semantic analysis and does not address how ownership is inferred once a program contains multiple interacting functions.

A third line of work does not attempt to prevent unsafe or non-compiling translations during code generation at all, and instead repairs Rust code after the fact. Yang et al. [8] introduce Rust-lancet, which applies an iterative, template-based repair framework directly to Rust source that already fails to compile because of an ownership-rule violation, using AST-level rewrites such as inserting a `clone()` call or reordering statements, and validates each fix with a behaviour-preservation check before accepting it. Ling et al. [9] apply, in a related but earlier approach, a fixed catalogue of source-to-source rewrite rules to the output of a syntactic transpiler specifically to reduce the number of function signatures that must be marked `unsafe`, without relying on compiler feedback. Both approaches are valuable as a safety net, and the LLM-guided repair agent proposed as part of the C2SafeRust-LLM pipeline occupies a similar position in the overall architecture. But repairing an already-broken or already-unsafe translation is a strictly more expensive strategy than avoiding the mistake in the first place: every one

of the interprocedural patterns considered in this project would, under an intra-procedural code generator, either fail to compile and consume a repair iteration or be conservatively marked unsafe, precisely the two outcomes that motivate looking further upstream in the pipeline.

Read together, these four strands of literature converge on the same structural observation from different directions. Emre et al. [4, 5] locate, across their compiler-feedback-driven translation work and its empirical follow-up study, the residual bottleneck in call-unaware aliasing reasoning. Hong and Ryu [6] demonstrate that treating one interprocedural pattern explicitly is both possible and effective, but does not generalise across idioms. Peng et al. [7] widen the accepted C subset without touching ownership inference. Yang et al. [8] and Ling et al. [9] accept ownership mistakes as a cost of doing business and clean them up afterwards. None of these treats interprocedural ownership transfer as a first-class analysis pass sitting between per-function semantic analysis and code generation, propagating alias and ownership summaries across the call graph before any Rust type is committed to. This is the gap the proposed extension to C2SafeRust-LLM is designed to close, and the environment required to build and evaluate it is described in the next chapter.

CHAPTER 3

ENVIRONMENTAL STUDY

This chapter describes the environment required to build, extend, and evaluate C2SafeRust-LLM, the single bundled extension comprising preprocessor expansion, multi-level pointer chain analysis, union-to-enum transformation, interprocedural analysis, and an LLM-guided repair agent. Because the extension is implemented inside the existing Rust-native C2SafeRust pipeline rather than as a separate tool, its environmental requirements are largely an extension of the base system’s requirements, with two additions this project introduces: a graph data-structure library capable of representing whole-program call graphs and alias summaries, needed for interprocedural analysis; and an HTTP client capable of reaching the new LLM repair agent’s API, needed because that agent is the fallback path for the one idiom – data-dependent ownership – that static interprocedural analysis cannot fully resolve.

3.1 System Configuration

The system configuration covers the hardware and software needed to compile the pipeline itself, to run it against multi-function C programs of realistic size, and to invoke the LLM repair agent when the interprocedural pass hands off a residual case. Because the new analysis pass must hold a call graph, a set of alias summaries, and a per-function ownership summary in memory simultaneously for the whole translation unit rather than one function at a time, the working-set size during semantic analysis is larger than in the intra-procedural system, which is the main additional consideration when specifying hardware.

3.1.1 *Hardware Requirements*

The key hardware requirements are detailed as follows:

- **Processor:** Intel® Core™ i5 processor or AMD Ryzen™ 5 or higher, sufficient for rustc compilation cycles and call-graph construction over multi-function benchmark programs.
- **Memory:** Minimum 8 GB RAM; 16 GB recommended when evaluating larger benchmark programs, since the call graph, alias summaries, and per-function ownership maps for an entire translation unit must be held in memory together during interprocedural analysis, rather than being discarded between functions as in the intra-procedural system.
- **Storage:** 50–100 GB of free disk space, for the Rust toolchain, cargo build artifacts, benchmark C programs, and generated translation reports.
- **Network:** A working internet connection (or access to a locally hosted equivalent), required when the repair loop escalates an unresolved data-dependent-ownership case to the LLM repair agent’s API.

3.1.2 *Software Requirements*

The key software requirements are detailed as follows:

- **Operating System:** Ubuntu 20.04 LTS or later, Windows 10/11, or macOS, since the pipeline and its dependencies are cross-platform.
- **Programming Language:** Rust (stable channel), used to implement every stage of the pipeline, including the new interprocedural analysis pass.
- **Build Tool:** Cargo, for dependency management and building the pipeline and its test benchmarks.
- **Version Control System:** Git, for managing the codebase and the benchmark suite used to evaluate the extension.

- **Required Rust Crates:**

- `serde` – deserialises the structured JSON diagnostics emitted by `rustc` during the compiler-feedback loop.
- `petgraph` – provides the graph data structures used to represent the whole-program call graph and the points-to/alias graph that the interprocedural pass builds and traverses.
- `request` – an HTTP client used by the repair loop to send `rustc` diagnostics to the LLM repair agent and receive proposed patches when a case cannot be resolved statically.
- `egui` – provides the graphical front end that wraps the pipeline and displays validation reports and translation output.

3.1.3 Software Requirements for Development

The key development software requirements are detailed as follows:

- **Operating System:** Ubuntu 20.04 LTS or later, Windows, or macOS, ensuring cross-platform development.
- **Back End:** Rust (stable channel), the sole implementation language for the pipeline.
- **IDE Used:** Visual Studio Code with the `rust-analyzer` extension, for code navigation, inline diagnostics, and debugging support during development of the interprocedural pass.
- **Version Control:** Git, for managing the source repository.
- **Compiler Toolchain:** `rustc`, invoked with `--error-format=json` by the compiler loop, both to validate translations produced with the new pass and to drive the LLM-guided repair loop this project introduces for cases the static passes do not resolve.

3.2 Software Specification

The following section describes the individual software components used to build and run the interprocedural extension, and the specific role each one plays in the pipeline.

3.2.1 Rust and Cargo

Rust is the implementation language for the entire pipeline, including the new interprocedural analysis pass, for the same reason it is the compilation target: writing the analyser itself in Rust keeps the toolchain self-hosted and avoids introducing a second language runtime purely for tooling. Cargo manages the crate’s dependencies – including *serde*, *petgraph*, *reqwest*, and *egui* – and builds both the pipeline and the benchmark harness used to evaluate it.

3.2.2 rustc and Structured Diagnostics

The Rust compiler, *rustc*, continues to serve two roles in the extended pipeline. First, it is the ground-truth validator for every candidate translation the interprocedural pass produces: a translation is only considered successful once it compiles. Second, invoked with its JSON diagnostic format, it remains the input to the compiler-feedback repair loop; the compiler-invocation mechanism is reused unchanged from the base system, but the repair engine consuming the diagnostics is now the LLM-guided agent this project introduces, serving as the fallback for the data-dependent-ownership idiom that the interprocedural pass cannot resolve statically.

3.2.3 petgraph

petgraph is the graph library used to represent the two structures the interprocedural pass depends on: a call graph, whose nodes are functions and whose edges are call sites, used to determine the order in which per-function ownership summaries must be computed; and an interprocedural alias graph,

whose edges track how a pointer argument at a call site relates to the variable it originates from in the caller. Representing both structures explicitly as graphs, rather than as ad hoc maps, is what allows the pass to detect aliasing that only exists at a particular call site – such as two arguments to the same call resolving to the same caller variable – which is precisely the situation an intra-procedural analysis of the callee alone cannot see.

3.2.4 *serde*

`serde` deserialises the structured JSON diagnostics that `rustc` emits, converting compiler error codes and their associated spans into a form the repair loop can act on. This deserialisation role is unchanged from the base system, even though this project replaces the rule-based engine consuming that output with the LLM-guided repair agent; the interprocedural pass is intended to reduce how often the repair loop needs to run for the six target idioms, not to replace it.

3.2.5 *reqwest*

`reqwest` provides the HTTP client used by the repair loop to communicate with the LLM repair agent's API when a translation cannot be resolved by the static passes – either because the interprocedural analysis was not designed to resolve it, as with the data-dependent-ownership idiom, or because an unrelated compiler error falls outside both the rule-based and interprocedural coverage.

3.2.6 *egui*

`egui` provides the immediate-mode graphical interface that wraps the pipeline, displaying the source program, the generated Rust output, and the translation report – including, for the interprocedural extension, a summary of which of the six target idioms were detected in the input program and how each was resolved.

In this chapter, the hardware, software, and Rust ecosystem components

required to build and evaluate the C2SafeRust-LLM extension have been described. With the environment established, the next chapter analyses the requirements of the proposed system as a whole in detail, including why C2SafeRust's strictly intra-procedural design cannot be extended to support the six target idioms without an interprocedural analysis pass.

CHAPTER 4

SYSTEM ANALYSIS

System analysis is the process of structurally examining the proposed C2SafeRust-LLM system, a single extension to the base C2SafeRust pipeline. Its purpose is to establish, precisely, which C ownership idioms the existing intra-procedural C2SafeRust pipeline cannot translate correctly, why each failure is a structural consequence of analysing one function at a time rather than a defect that could be fixed by refining per-function analysis alone, and what the new interprocedural analysis pass – together with the other components bundled into this proposal – must do to resolve them before code generation.

The analysis in this chapter follows the six-idiom taxonomy developed for this project: aliased mutable parameters, ownership consumed through a T** argument, ownership created through a T** out-parameter, overlapping pointer-arithmetic slices at a call site, a union’s active variant carried across a call boundary, and data-dependent ownership. Each idiom is traced to the specific new component whose per-function scope would leave it unresolved, and each functional requirement in this chapter is traceable back to the idiom that motivates it.

4.1 Requirements Analysis

This chapter’s requirements analysis covers the C2SafeRust-LLM system as a whole – the preprocessor expansion stage, the multi-level pointer chain analyser, the union-to-enum transformer, the new interprocedural analysis pass, and the LLM-guided repair agent – while giving particular depth to the interprocedural pass, since it is what makes the other four components’ output

translatable to safe Rust across function boundaries. The interprocedural pass must be able to construct a call graph for an entire C translation unit, propagate ownership and alias information for each function as a reusable summary rather than recomputing it per caller, and integrate its output into the `OwnershipKind` classification step without requiring changes to the code generator’s type-mapping rules. It must also recognise the one idiom it cannot fully resolve – data-dependent ownership – and route that case to a conservative fallback rather than silently mistranslating it.

4.2 Existing System

The existing system is C2SafeRust, the base compiler-in-the-loop pipeline this project extends. It performs semantic analysis strictly per function: for each function, a symbol table is built, def-use chains are computed, an alias graph is constructed over the pointers used within that function body, memory-access patterns are classified, and an ownership category is inferred for every variable, all without reference to how the function is called or what its arguments alias at any particular call site. C2SafeRust accepts only a single-level-pointer C subset – scalar expressions, struct field access, one-dimensional array indexing, nested loops, and recursion – and its input validator rejects, at Stage 1, any source file containing `T**` or deeper pointer chains, C unions, preprocessor macros or conditional-compilation directives, or more than one translation unit. The compiler-feedback repair loop is strictly rule-based: a four-pass `RepairEngine` covering roughly a dozen hardcoded `rustc` error codes, and it is the system’s only mechanism for handling anything the per-function analysis gets wrong, operating strictly after code generation, once `rustc` has already rejected the candidate translation.

4.3 Limitations of Existing System

C2SafeRust’s per-function analysis scope, on its own, already fails on aliased mutable parameters (idiom 1 below) even without any new component. But the remaining five idioms only become visible once the C

subset is widened at all – since C2SafeRust’s input validator rejects T^{**} , unions, and macros outright rather than mistranslating them. The argument in this section is therefore forward-looking: it establishes that a per-function version of the multi-level pointer chain analyser and the union-to-enum transformer – the very components this project is adding – would still fail on five of these six idioms if implemented without also adding interprocedural analysis. This is why the interprocedural pass cannot be deferred to a later phase; it has to be designed in alongside the other new components from the start, summarised in Table 4.1. In each case, the information needed to make the correct ownership decision exists at a call site or inside a different function’s body than the one currently being analysed, so no amount of additional per-function analysis – however capable the underlying component – can resolve it.

- **Aliased mutable parameters:** two pointer parameters that look independent inside a function body may alias at a specific call site. Translating both as independent $\&\text{mut } T$ borrows is only unsound at the call sites where the arguments actually alias, a fact the callee’s body cannot express. Even C2SafeRust’s existing single-level-pointer analysis emits code that `rustc` rejects with error E0499 in this situation, consuming a repair iteration for a case that a call-site alias check could have caught before code generation.
- **Ownership consumed through a T^{**} argument:** a function that frees the pointee of a T^{**} parameter and nulls it out is relinquishing ownership on behalf of its caller. A per-function pointer-chain analyser, even once extended to track multi-level pointers, has no mechanism to link that write-through-pointer effect back to the caller’s local variable, since doing so requires opening the callee’s body while analysing the caller.
- **Ownership created through a T^{**} out-parameter:** the mirror image of the previous case, and an extremely common systems-programming idiom. Only the callee’s body proves that the value written through the

out-parameter is newly allocated and owned by the caller afterward; the caller's own body gives no such indication, so a per-function pointer-chain analyser cannot resolve this either.

- **Overlapping pointer-arithmetic slices at a call site:** a function signature such as `combine(int *dst, const int *src, int n)` looks safe in isolation, translating naturally to `&mut [i32]` and `&[i32]`. Whether that translation is sound depends entirely on the offsets used at each call site, which the callee's body does not contain – so, again, a per-function pointer-chain analyser cannot decide this alone.
- **A union's active variant carried across a call boundary:** recognising which field of a C union is live at a given access requires anchoring to the most recent write to that union. A per-function union-to-enum transformer would have no prior write to anchor to when the write happens in one function and the read happens in another.
- **Data-dependent ownership:** in the hardest case, whether a function takes ownership of its argument depends on a run-time value, not on any static call-site property. This case is included in the taxonomy specifically because it marks the boundary of what an interprocedural static pass can achieve; it is expected to remain a case for this project's LLM-guided repair agent, or for a conservative `Option<Box<T>>` fallback, rather than for full static resolution.

4.4 Proposed System

The proposed system, C2SafeRust-LLM, bundles five components into a single extension of the base C2SafeRust pipeline: a preprocessor expansion stage that resolves macros and conditional-compilation directives so a wider C subset can be parsed at all; a multi-level pointer chain analyser that tracks `T**` and deeper pointer chains; a union-to-enum transformer that converts C unions with a discriminable active variant into safe Rust enums; a new

Table 4.1: Traceability from idiom to the new component that is insufficient if scoped per-function, and the interprocedural mechanism proposed to resolve it

#	Idiom	Insufficient if scoped per-function	Proposed resolution
1	Aliased <code>&mut</code> parameters	Base per-function alias analysis	Interprocedural points-to/alias analysis over call-site arguments
2	<code>T**</code> consumes ownership	Multi-level pointer chain analyser (new)	Def-use link from callee’s write-through pointer back to caller’s variable
3	<code>T**</code> creates ownership	Multi-level pointer chain analyser (new)	Same mechanism, reversed direction (callee $\rightarrow$ caller)
4	Overlapping pointer-arithmetic slices	Multi-level pointer chain analyser (new, pointer arithmetic)	Call-site range/offset overlap check
5	Union variant set in another function	Union-to-enum transformer (new)	Interprocedural reaching-definitions over the tag/variant state
6	Data-dependent (run-time) ownership	Interprocedural analysis (fundamental limit)	Path-sensitive analysis, or fallback to iterative LLM-guided repair

interprocedural analysis pass; and an LLM-guided repair agent that replaces C2SafeRust’s rule-based `RepairEngine`. The remainder of this section details the interprocedural pass, since it is the mechanism that lets the other four components’ output be classified correctly across function boundaries. The pass is positioned after per-function semantic analysis and before ownership classification, and operates over the whole translation unit rather than one function at a time. It builds a call graph from every direct call in the program; computes, for each function, a summary of how its pointer parameters relate to values returned or written through other parameters; propagates these summaries along call-graph edges so that a caller’s alias and ownership analysis can consult what is known about its callees without re-deriving it from scratch; and performs a call-site-specific check for aliasing and slice-overlap conditions using the actual arguments passed at each call, not just the parameter types declared by the callee. The output of this pass is consumed by the `OwnershipInferencer` as additional input, so that a variable’s final

`OwnershipKind` reflects both its intra-procedural usage and any call-site conditions discovered interprocedurally, without changing how the code generator maps a given `OwnershipKind` to a Rust type.

4.5 Advantages of Proposed System

The proposed system offers the following advantages over the existing intra-procedural pipeline:

- Ownership decisions that depend on call-site information are resolved before code generation, rather than being discovered only after `rustc` rejects the output, which is expected to reduce the number of repair iterations consumed by the five statically-resolvable idioms.
- Fewer conservative `unsafe` classifications are expected for write-through pointer parameters, since ownership-transfer direction can be determined from the callee’s body rather than assumed defensively.
- The single idiom that cannot be resolved statically is identified explicitly rather than silently mistranslated, and is routed to this project’s LLM-guided repair agent or a conservative fallback type, preserving the correctness guarantees of the rest of the pipeline.
- The interprocedural summaries are computed once per function and reused at every call site, rather than being recomputed per caller, keeping the added analysis cost proportional to the size of the call graph rather than to the number of call sites.

4.6 Functional Requirements

This section describes the specific operations the C2SafeRust-LLM system must perform, covering all five bundled components; each requirement is traceable to one or more of the six idioms in Table 4.1 where applicable.

4.6.1 *Input File Acceptance and Validation*

The system shall accept a single C source file as input, verify that it is readable and encoded correctly, and reject the file with a descriptive error message if it cannot be lexed or parsed under the accepted C subset, before any further pipeline stage is invoked. This precedes preprocessor expansion and AST processing and prevents downstream stages from operating on malformed input.

4.6.2 *Preprocessor Expansion*

The system shall resolve C preprocessor macros and conditional-compilation directives before AST construction, so that source files widened beyond C2SafeRust's original macro-free subset can be parsed at all. This is a precondition for the rest of the pipeline being able to see the true source text of a program rather than rejecting it at Stage 1 as C2SafeRust would.

4.6.3 *AST Construction*

The system shall parse a preprocessor-expanded, validated input file into an internal abstract syntax tree representing every function, statement, and expression in the translation unit. This AST is the shared input to per-function semantic analysis, the new interprocedural analysis stage, and Rust code generation, and must therefore preserve source-level structure (function boundaries, pointer expressions, call sites) needed by all three.

4.6.4 *Multi-Level Pointer Chain Analysis*

The system shall track pointer chains of depth two or greater (T** and beyond) during semantic analysis, classifying whether a function frees, reassigns, or newly allocates the pointee at each level of indirection. This is a precondition for idioms 2 and 3, and its output is what the "Ownership-Transfer Summaries for Multi-Level Pointers" requirement below propagates across call sites.

4.6.5 Union-to-Enum Transformation

The system shall identify C unions with a discriminable active-variant pattern (for example, a companion tag field) and transform them into safe Rust enums, resolving the active variant at each access site it can determine locally. This is a precondition for idiom 5, whose cross-function cases are resolved by the “Cross-Function Union Variant Tracking” requirement below.

4.6.6 Rust Code Generation

The system shall generate a candidate Rust translation from the AST, the per-function semantic model, and the ownership classifications produced by the OwnershipInferer — including any classifications revised by the interprocedural pass — and shall write the generated source to an output .rs file. Code generation shall not require any change to its existing type-mapping rules to consume interprocedural output, consistent with the non-functional maintainability requirement in Section 4.7.4.

4.6.7 Call Graph Construction

The system shall construct a call graph for the entire translation unit, with one node per function and one edge per call site. This is a precondition for resolving idiom 5, and for correctly ordering the computation of per-function summaries so that a function’s summary is available before its callers are analysed.

4.6.8 Interprocedural Alias and Points-to Analysis

The system shall compute, for each call site, whether any two pointer arguments can refer to the same underlying storage, using the actual expressions passed at that call site rather than the callee’s parameter types alone. This directly addresses idiom 1 and the call-site-specific overlap condition in idiom 4.

4.6.9 Ownership-Transfer Summaries for Multi-Level Pointers

The system shall compute, for each function that takes a T^{**} parameter, whether the function frees or reassigns the pointee (idiom 2) or allocates and writes a new value through it (idiom 3), and shall propagate this as a summary to every call site so the caller's local variable can be reclassified accordingly.

4.6.10 Cross-Function Union Variant Tracking

The system shall extend the existing per-access-site union flow analysis to an interprocedural reaching-definitions analysis, so that a union's active variant can be determined at a read site even when the most recent write to that union occurred in a different function. This addresses idiom 5.

4.6.11 Fallback for Data-Dependent Ownership

The system shall detect the case where ownership depends on a run-time value rather than any static call-site property, and shall emit a conservative `Option<Box<T>>` translation or escalate the case to the LLM-guided repair agent described below, rather than attempting a static resolution that could be silently unsound. This addresses idiom 6 and is a deliberate non-goal for the static analysis pass itself.

4.6.12 LLM-Guided Repair Agent

The system shall replace C2SafeRust's rule-based `RepairEngine` with an LLM-guided repair agent that accepts `rustc`'s structured JSON diagnostics for any case the preprocessor expansion, pointer-chain analysis, union-to-enum transformation, and interprocedural analysis stages did not preempt, together with the original C source corresponding to the function(s) under repair, proposes a candidate patch, and re-verifies the patch by recompilation before accepting it. Supplying the original C source alongside the compiler diagnostics allows the agent to recover ownership intent that is not observable from the generated Rust or its diagnostics alone – information the base

system's rule-based engine never had access to, since it operated on `rustc`'s output in isolation. This is particularly relevant for the data-dependent-ownership idiom, where the diagnostics describe only a symptom in the generated Rust while the C source is the only remaining record of what the original ownership transfer was meant to do. This generalises repair coverage beyond the fixed error-code list a rule-based engine is limited to, and serves as the fallback path for idiom 6.

4.7 Non-Functional Requirements

The non-functional requirements specify the quality attributes the C2SafeRust-LLM extension as a whole must satisfy so that adding its five bundled components does not degrade the usability or reliability of the existing C2SafeRust pipeline.

4.7.1 Scalability

The pass shall scale to translation units containing multiple interacting functions without recomputing a callee's summary once per caller; summaries shall be computed once per function and reused across all of that function's call sites, so that analysis cost grows with the size of the call graph rather than with the number of call-site instances.

4.7.2 Backward Compatibility

The pass shall not change the Rust type mapping for any program that does not exhibit one of the six target idioms; single-function programs and programs with no aliasing across call boundaries shall be translated identically to how the existing intra-procedural system translates them.

4.7.3 Transparency

Every ownership decision the interprocedural pass changes from what the intra-procedural analysis alone would have produced shall be recorded in the existing `TranslationReport`, identifying which of the six idioms triggered

the change, consistent with how the existing repair engine already records every `RepairAction` it applies.

4.7.4 Maintainability

The pass shall be implemented as a distinct module consuming the existing per-function `SemanticModel` and producing summaries consumed by the existing `OwnershipInferencer`, so that it can be tested, and its coverage of the six idioms evaluated, independently of the code generation and repair stages.

4.8 Feasibility Study

This section evaluates the feasibility of the proposed extension from technical, operational, and economic perspectives.

4.8.1 Technical Feasibility

The pass builds directly on infrastructure the existing pipeline already has: a per-function semantic model, an alias graph, and a memory-pattern analyser. Call-graph construction and interprocedural summary propagation are well-understood static-analysis techniques, and the graph data structures required for them are available as mature Rust libraries. Five of the six target idioms are addressed by static analysis of this kind; the sixth is explicitly deferred to this project's LLM-guided repair agent rather than treated as a technical risk.

4.8.2 Operational Feasibility

The extension requires no change to how a user invokes the pipeline; the interprocedural pass runs automatically as part of semantic analysis, and its output is visible in the existing translation report without any additional configuration.

4.8.3 *Economical Feasibility*

The pass itself adds no new paid dependency or external service; the graph libraries required are open-source Rust crates. The one external dependency introduced by this project as a whole is the LLM-guided repair agent's API call – the base C2SafeRust system has no network dependency at all, since its repair engine is purely rule-based.

This analysis has identified six concrete idioms that the existing intra-procedural C2SafeRust pipeline cannot translate correctly, traced each one to the new component whose per-function scope would leave it unresolved, and derived the functional and non-functional requirements for a bundled extension – preprocessor expansion, multi-level pointer chain analysis, union-to-enum transformation, interprocedural analysis, and an LLM-guided repair agent – that resolves five of the six idioms statically while explicitly deferring the sixth to the LLM-guided repair agent. This establishes the requirements baseline for the system design presented in the following chapter.

CHAPTER 5

SYSTEM DESIGN

This chapter details the system design of the C2SafeRust-LLM transpilation framework. It presents the high-level application architecture, data flow diagrams across Level 0, Level 1, and Level 2, and a comprehensive functional breakdown of the underlying modules. The design bridges low-level C semantics with Rust’s strict compile-time safety and ownership requirements by combining static semantic analysis with an iterative, compiler-validated LLM repair loop.

5.1 *Application Architecture*

The system architecture of C2SafeRust-LLM governs the end-to-end transformation of C source code into safe, compilable, and idiomatic Rust code through twelve interconnected functional blocks. The pipeline begins at the front-end, where user-provided C source files are accepted, validated for syntactic sanity, preprocessed to expand macros and header inclusions, and parsed into a location-aware Abstract Syntax Tree (AST) capturing full source metadata (Blocks 1–4).

Following AST construction, the system enters its analytical core by combining per-function semantic analysis with whole-program interprocedural static analysis. The intraprocedural pass extracts symbol tables, analyzes multi-level pointer chains (T^{**}), and handles local union-to-enum transformations, while the interprocedural pass constructs global call graphs, evaluates call-site parameter aliasing, tracks cross-function union variant usage, and builds ownership-transfer summaries across function boundaries (Blocks 5–6).

Using these combined semantic insights, the framework infers optimal Rust ownership types (`Box<T>`, `&mut T`, `Vec<T>`) and synthesizes candidate Rust source code (`.rs`) designed to minimize unsafe block usage (Blocks 7–8). To guarantee correctness, this candidate code is submitted to the `rustc` compiler in JSON diagnostic output mode. If compilation errors or borrow checker violations occur, an LLM-guided repair agent packages the JSON feedback, queries an external repair API, applies proposed code patches, and re-verifies the output iteratively until compilation succeeds (Blocks 9–10).

Finally, decision logs, repair histories, and execution metrics are aggregated into a translation report, presenting the final verified Rust code alongside the detailed report to the user through the graphical interface (Blocks 11–12). Figure 5.1 illustrates the overall block architecture.

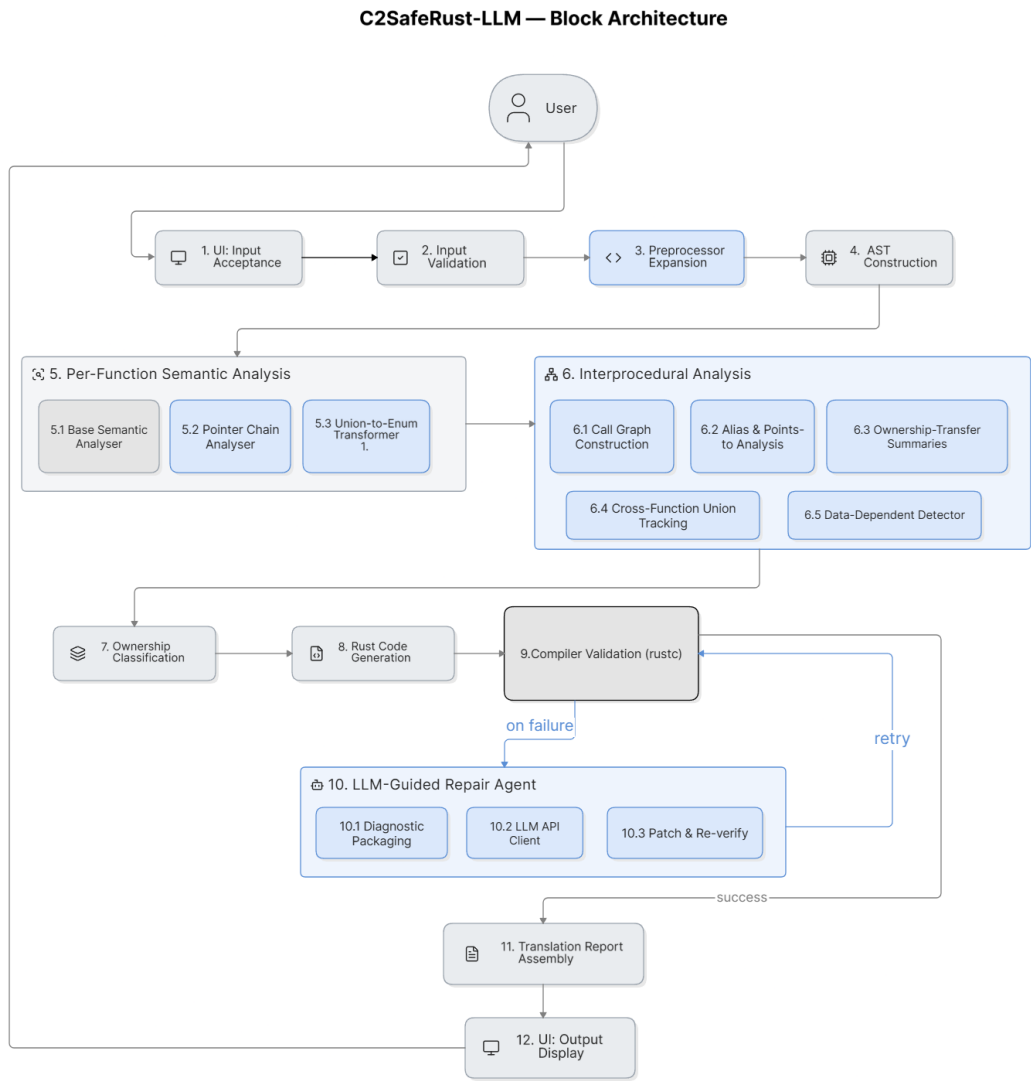


Fig. 5.1: C2SafeRust-LLM — Block Architecture

5.2 Data Flow Diagram

The Data Flow Diagrams (DFDs) describe the movement, transformation, and storage of data across C2SafeRust-LLM from initial input to final report display.

5.2.1 DFD Level 0

Figure 5.2 shows the Level 0 Context Diagram of the C2SafeRust-LLM system. The user interacts directly with process 0.0 by supplying a C source file (.c) and receiving the final Rust code (.rs) along with a comprehensive translation report.

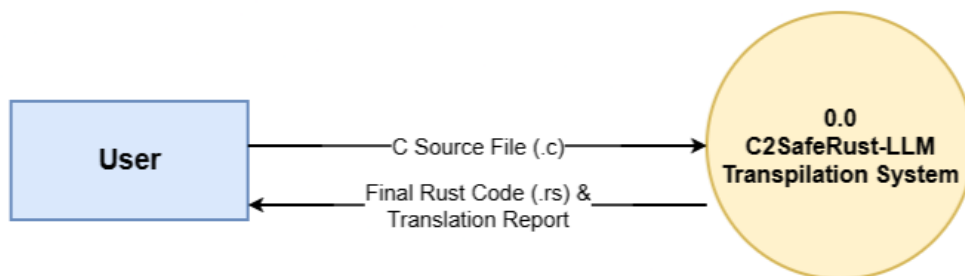


Fig. 5.2: Data Flow Diagram Level 0 (Context Diagram)

5.2.2 DFD Level 1

Figure 5.3 decomposes process 0.0 into seven primary processing stages, two persistent data stores (**D1 Semantic Models** and **D2 Interprocedural**

Summaries), and three external entities (User, rustc Compiler, and LLM Repair API).

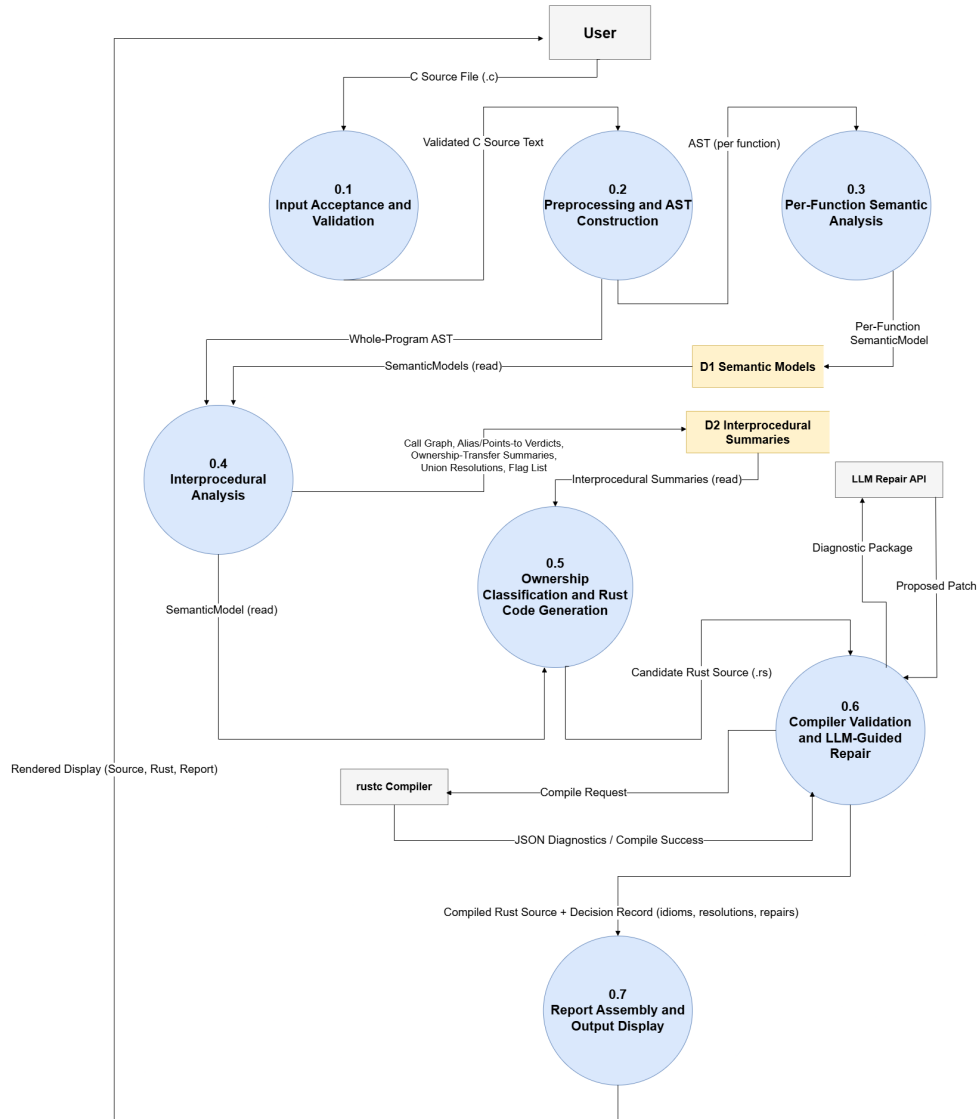


Fig. 5.3: Data Flow Diagram Level 1

The Level 1 data flows operate as follows:

- **0.1 Input Acceptance and Validation:** Receives the C source file from the User, verifies validity, and passes Validated C Source Text to process 0.2.

- **0.2 Preprocessing and AST Construction:** Parses validated C code into per-function ASTs (sent to 0.3) and a Whole-Program AST (sent to 0.4).
- **0.3 Per-Function Semantic Analysis:** Analyzes per-function ASTs and stores intraprocedural models into **D1 Semantic Models**.
- **0.4 Interprocedural Analysis:** Reads per-function models from **D1** alongside the Whole-Program AST to generate call graphs, aliasing verdicts, ownership transfer summaries, and flag lists, storing results in **D2 Interprocedural Summaries**.
- **0.5 Ownership Classification and Rust Code Generation:** Reads models from **D1** and summaries from **D2** to generate candidate Rust source code (.rs).
- **0.6 Compiler Validation and LLM-Guided Repair:** Sends compile requests to the rustc Compiler. On compilation failure, dispatches diagnostic packages to the LLM Repair API to retrieve patch proposals. Upon compilation success, forwards compiled Rust source and decision records to process 0.7.
- **0.7 Report Assembly and Output Display:** Assembles the final output display containing source C, generated Rust, and translation reports, returning them to the User.

5.2.3 DFD Level 2

Level 2 DFDs expand the analytical and repair processes (0.3, 0.4, and 0.6) into detailed sub-process data flows.

5.2.3.1 Expansion of Process 0.3: Per-Function Semantic Analysis

Figure 5.4 illustrates the decomposition of process 0.3 into three sub-processes populating store **D1 Semantic Models**.

- **0.3.1 Base Semantic Analyser:** Processes ASTs to generate Symbol Tables, Def-Use Chains, Alias Graphs, and Memory-Access

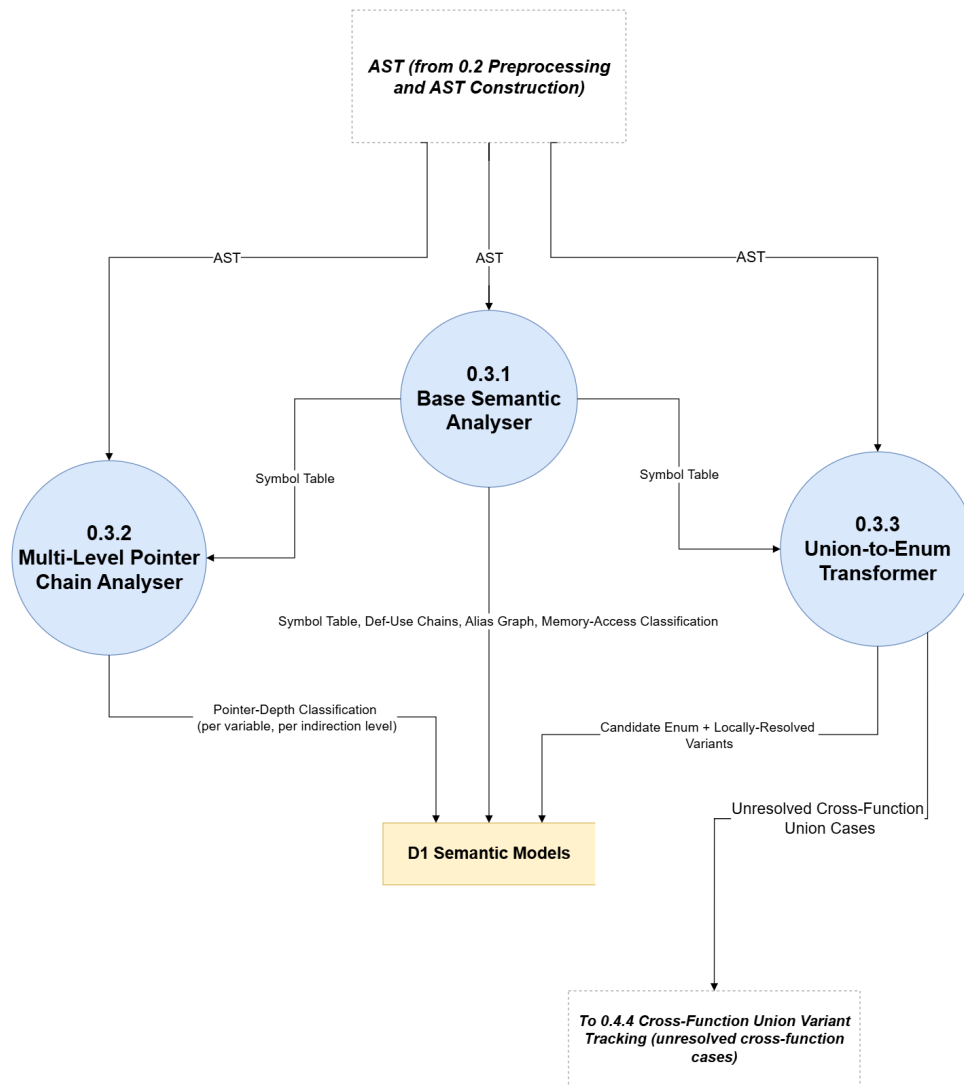


Fig. 5.4: Data Flow Diagram Level 2 — Expansion of Process 0.3

Classifications stored into **D1 Semantic Models**.

- **0.3.2 Multi-Level Pointer Chain Analyser:** Consumes ASTs and symbol tables from 0.3.1 to generate Pointer-Depth Classifications (per variable, per indirection level) stored in **D1 Semantic Models**.
- **0.3.3 Union-to-Enum Transformer:** Consumes ASTs and symbol tables from 0.3.1 to generate candidate enums and locally resolved variants saved in **D1**. Unresolved cross-function union cases are forwarded to process 0.4.4.

5.2.3.2 Expansion of Process 0.4: Interprocedural Analysis

Figure 5.5 details the expansion of process 0.4 into five interprocedural analysis units populating store **D2 Interprocedural Summaries**.

- **0.4.1 Call Graph Construction:** Processes Whole-Program AST to build a global call graph dispatched to processes 0.4.2, 0.4.3, 0.4.4, and 0.4.5.
- **0.4.2 Alias and Points-to Analysis:** Combines call graphs and call-site argument expressions to compute per-call-site aliasing verdicts saved in **D2 Interprocedural Summaries**.
- **0.4.3 Ownership-Transfer Summaries:** Evaluates call graphs and T^{**} indirection classifications from **D1** to derive ownership-transfer summaries written to **D2** and passed to 0.4.5.
- **0.4.4 Cross-Function Union Variant Tracking:** Tracks unresolved union cases from 0.3.3, call graphs, and local union resolutions from **D1** to store cross-function union resolutions in **D2** and send summaries to 0.4.5.
- **0.4.5 Data-Dependent Ownership Detector:** Correlates

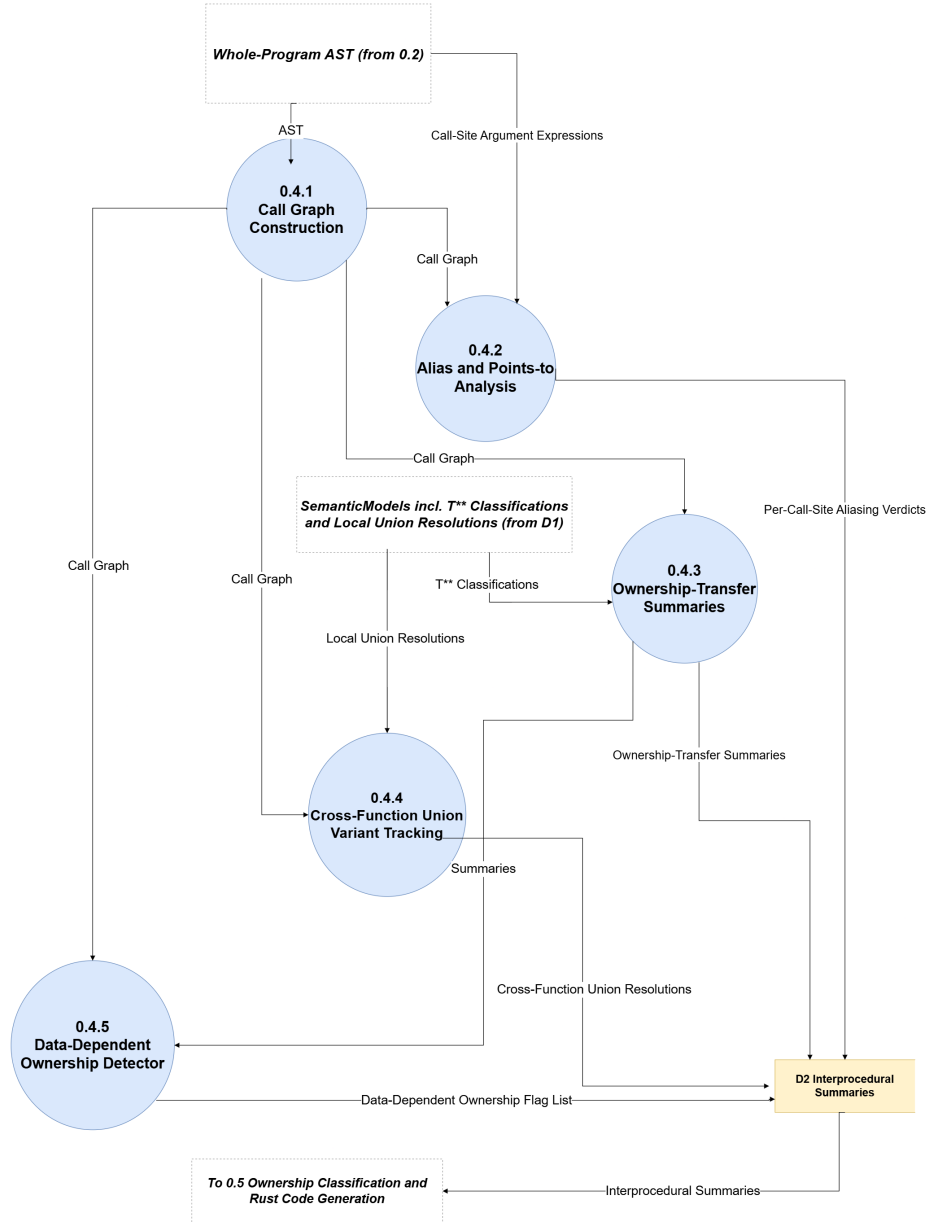


Fig. 5.5: Data Flow Diagram Level 2 — Expansion of Process 0.4

5.2.3.3 Expansion of Process 0.6: Compiler Validation and LLM-Guided Repair

Figure 5.6 details the expansion of process 0.6 into four interactive feedback and repair modules. call graphs with summaries from 0.4.3 and 0.4.4 to output data-dependent ownership flag lists into **D2 Interprocedural Summaries**.

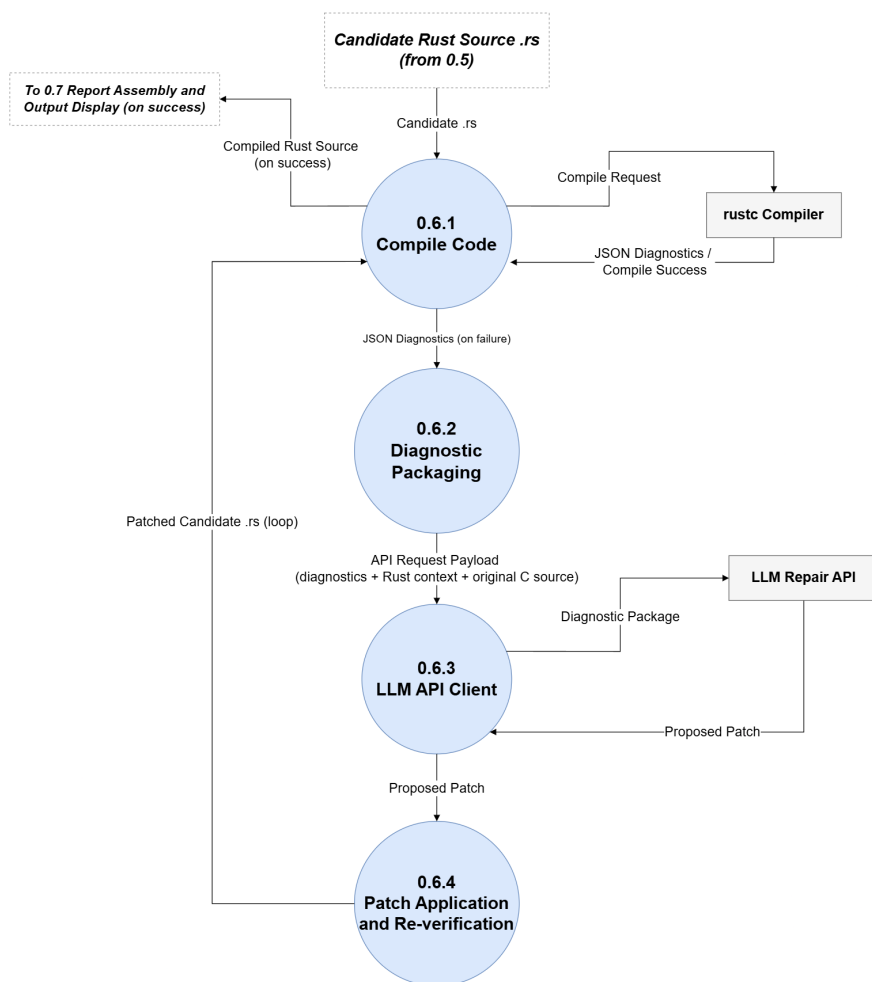


Fig. 5.6: Data Flow Diagram Level 2 — Expansion of Process 0.6

- **0.6.1 Compile Code:** Submits candidate or patched Rust source to the external `rustc` Compiler. Upon successful compilation, outputs

compiled Rust source to process 0.7. Upon failure, routes JSON diagnostics to process 0.6.2.

- **0.6.2 Diagnostic Packaging:** Combines JSON diagnostics, original C source, and Rust context into a structured API request payload for process 0.6.3.
- **0.6.3 LLM API Client:** Transmits diagnostic packages to the external LLM Repair API and receives proposed code patches.
- **0.6.4 Patch Application and Re-verification:** Applies proposed patches to candidate source code and routes patched Rust code back to process 0.6.1 for re-compilation.

5.3 List of Modules

The C2SafeRust-LLM transpilation framework comprises seven primary functional modules:

5.3.1 Input Acceptance and Validation Module

Responsible for accepting input C source files (.c), validating syntax, performing preliminary checks for unsupported constructs, and providing structured pre-flight validation status to the pipeline.

5.3.2 Preprocessing and AST Construction Module

Handles macro expansion, preprocessor directives, and header inclusions, building location-aware Abstract Syntax Trees (ASTs) that capture structural, expression, and statement metadata.

5.3.3 Per-Function Semantic Analysis Module

Executes intraprocedural semantic analysis across individual function bodies. It constructs symbol tables, def-use chains, alias graphs, multi-level pointer-depth classifications, and local union-to-enum transformations stored in **D1 Semantic Models**.

5.3.4 Interprocedural Analysis Module

Executes program-wide static analysis. It builds global call graphs, resolves cross-function pointer aliasing, derives ownership-transfer summaries, tracks cross-function union variant usage, and identifies data-dependent ownership boundaries stored in **D2 Interprocedural Summaries**.

5.3.5 Ownership Classification and Code Generation Module

Infers optimal Rust ownership categories (`Box<T>`, `&mut T`, `&T`, `Vec<T>`) using models from **D1** and summaries from **D2**. It synthesizes formatted candidate Rust source code (`.rs`) while minimizing unsafe blocks.

5.3.6 Compiler Validation and LLM-Guided Repair Module

Implements the compiler-in-the-loop repair cycle. It invokes `rustc` with JSON diagnostic flags, packages diagnostic feedback, queries the external `LLM Repair API` for candidate patch proposals, and re-verifies patched code until compilation succeeds.

5.3.7 Report Assembly and Output Display Module

Collects decision histories, repair logs, idiom transformations, and execution outputs into a comprehensive translation report, rendering C source code and generated Rust code side-by-side in the user interface.

BIBLIOGRAPHY

- [1] S. Klabnik and C. Nichols, *The Rust Programming Language*, 2nd ed. San Francisco, CA, USA: No Starch Press, 2023.
- [2] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang, “Understanding memory and thread safety practices and issues in real-world Rust programs,” in *Proc. 41st ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2020, pp. 763–779, doi: 10.1145/3385412.3386036.
- [3] N. D. Matsakis and F. S. Klock, “The Rust language,” in *Proc. ACM SIGAda Annual Conf. (HILT)*, 2014, pp. 103–104.
- [4] M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf, “Translating C to safer Rust,” *Proc. ACM Program. Lang.*, vol. 5, no. OOPSLA, Art. no. 121, Oct. 2021, doi: 10.1145/3485498.
- [5] M. Emre, P. Boyland, A. Parekh, R. Schroeder, K. Dewey, and B. Hardekopf, “Aliasing limits on translating C to safe Rust,” *Proc. ACM Program. Lang.*, vol. 7, no. OOPSLA1, Art. no. 94, pp. 551–579, Apr. 2023, doi: 10.1145/3586046.
- [6] J. Hong and S. Ryu, “Don’t write, but return: Replacing output parameters with algebraic data types in C-to-Rust translation,” *Proc. ACM Program. Lang.*, vol. 8, no. PLDI, Art. no. 176, Jun. 2024, doi: 10.1145/3656406.
- [7] H. Peng, B. Kasikci, G. L. Bernstein, and M. D. Ernst, “Hayroll: A modular wrapper for translating C macros and conditional compilation to Rust,” *Proc. ACM Program. Lang.*, vol. 10, no. PLDI, Art. no. 198, Jun. 2026, doi: 10.1145/3808276.

- [8] W. Yang, L. Song, and Y. Xue, “Rust-lancet: Automated ownership-rule-violation fixing with behavior preservation,” in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, 2024, pp. 85:1–85:13, doi: 10.1145/3597503.3639103.
- [9] M. Ling, Y. Yu, H. Wu, Y. Wang, J. R. Cordy, and A. E. Hassan, “In Rust we trust: A transpiler from unsafe C to safer Rust,” in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng.: Companion Proc. (ICSE-Companion)*, 2022, pp. 354–355, doi: 10.1145/3510454.3528640.